PROJECT 2

SMS Spam Classifier(Natural language Processing)

(MINOR)

Natural language processing (**NLP**) is a branch of artificial intelligence that helps computers understand, interpret and manipulate human language . To Build a SMS Spam Classifier using Machine Learning in Python. We will use **algorithms** such as :-

For this project, we will use **python libraries** like:-

- NLTK (Natural Language Toolkit)
- Scikit-Learn
- Pandas
- Multinomial Naive Bayes Classifier

Approach :-

1. **Reading the data :** To read the data, import the Pandas library and read the data using *pd.read_csv()* .Here the data in the file is Tab (\t) separated, so we must provide the "*sep*" (separate) parameter. Also, the file does not contain any column names, so we should provide column names using "*names*" parameter. Data is getting stored in a DataFrame named "*messages*". To view the first 5 rows of data, we should use *messages.head()*

We can see here that the first column contains the **Labels (dependent variable)** i.e., a message is Spam or Ham and the second column contains the actual **Messages (independent variable)**

2. Exploratory Data Analysis

We can see here that there are 5572 rows and 2 columns. It means that there are 5572 messages and 2 columns named "*Label*" and "*Message*". There are no missing values in the data.

.*value_counts()* helps to return total counts for each Category i.e. 'Ham' and 'Spam' . We can see that the ham messages are more than the Spam messages.

3. Data Pre-processing

Here, we are calculating the Length and Punctuation of each message for further analysis, and this is added to DataFrame (*messages*) as Column.

3. Text Cleaning

Messages contain text but with many punctuation, Stop-words (These are the words in any language which does not add much meaning to a sentence. They can safely be ignored without sacrificing the meaning of the sentence), special characters, and many forms of verb.

Now we will clean messages by removing the unnecessary things.

We will import **re** (regex library) and from nltk library, we will import **stopwords** and **WordNet Lemmatizer** (One such method of Lemmatization) and create its object.

Now we will iterate through every message, and using regex (substitute method), we will take everything from the message except small alphabets(a-z) and capital alphabets(A-Z) and substitute it with blank space. Next, we will lowercase the message (as abc is not the same as ABC) to make learning easy for the machine and then split it into words. Will pass split words in a list comprehension where we will check each word, that whether it exists in the stopwords collection by nltk , and if the word is not in stopwords, it will be lemmatized using WordNet Lemmatizer object. After each word is lemmatized, we will again join the words and form a sentence and append in the corpus list of sentences

Replacing the messages with the cleaned messages that is available in the corpus.

4. Model Building

Splitting into '*X*' , which contain the **independent variables** i.e., messages and '*y*' include **dependent variable** (target variable) i.e., labels (spam or ham)

```
X = messages['Message']
```

```
X.head()
```

```
0    go jurong point crazy available bugis n great ...
1                        ok lar joking wif u oni
2    free entry wkly comp win fa cup final tkts st ...
3                  u dun say early hor u c already say
4                  nah think go usf life around though
Name: Message, dtype: object
```

```
y = messages['Label']
```

```
y.head()
```

```
0     ham
1     ham
2    spam
3     ham
4     ham
Name: Label, dtype: object
```

4.1 Train Test Split

Using the scikit-learn library, we can split the data into train and test. Here I have split the data into 77% (training data) and 33% (testing data)

4.2 Dealing with Text (Natural Language data) data

Now it's time to talk about how to deal with the text data. We can't directly pass the text to the machine learning model as the machine only understands data in the form of 0's and 1's.

To solve this problem, we will use the concept of TF-IDF Vectorizer (Term Frequency-Inverse Document Frequency). It is a standard algorithm to transform the text into a meaningful representation of numbers and is used to fit the machine algorithm for prediction.

We can also use Count Vectorizer (Bag of Words), but Count Vectorizer does not put weights on words, unlike TF-IDF vectorizer.

We can use the TF-IDF vectorizer from the scikit-learn library. Next, create an object of TF-IDF vectorizer and fit_transform to the data, which will convert into a matrix of words and sentences.

Here, 3733 are the sentences of the X_train, and 5772 are the total words obtained from the sentences.

Multinomial Naive Bayes Classifier

We will import the MultinomialNB model from the scikit-learn library where we first provided *TfidfVectorizer()* object and then *MultinomialNB()* object. It should be provided in a sequence as

we want that firstly TfidfVectorizer should be executed and the output of it will be provided to the model, and at last, we fit the model with X_train and y_train.

To make a prediction, we need to pass the X_test data, and the Pipeline object will handle it, i.e., automatically vectorize it and make predictions for us.

We must know that accuracy itself is not capable of justifying that the model is working fine. We will use scikit-learn library to get a report on **confusion_matrix .**

Here we can see that “ham” label got predicted good but “spam” label prediction is not fine , so we can’t say that model is excellent. Model is lacking in predicting spam accurately. Created model named from a pipeline object that performs TfidfVectorizer and MultinomialNB Model then fit the X_train and y_train to the model. The predictions of the X_test using model and accuracy is approx **98.24%**. Now let’s also see the **evaluation metrics** to get a better insight into how the model is performing. We can see that “ham” got predicted amazing, and also the “spam” label prediction increased as compared to the “MultinomialNB” model.

Now the model is being created, let’s try out this on a custom text and see what the model predicts!